Chapter 3: ITS-303 Mock Test 2

Question 1:
How many stars will the following snippet print to the monitor?

```
i = 0
while i <= 5:
    i += 1
    if i % 2 == 0:
        break
    print('*')
```

A. zero
B. two
C. one
D. three

Answer: C.

Explanation

Topics: `while` `break`

```
i = 0
while i <= 5:
    print('1. i: ', i)  # 0 -> 1
    i += 1
    print('2. i: ', i)  # 1 -> 2
    if i % 2 == 0:  # False, True
        break
    print('*')  # *
```

i % 2 == 0 is known as a test for even numbers. In the first iteration **i** is an odd **1** and therefore the break does not get triggered and a star gets printed. In the second iteration **i** is an even **2** the break gets triggered and there are no more stars.

Question 2:
Which of the following statements is false?

A. The None value can not be used as an argument of arithmetic operators.
B. The None value may not be used outside functions.
C. The None value can be compared with variables.
D. The None value can be assigned to variables.

Answer: B.

Explanation

Topic: `None`

```
# The None value can be assigned to variables.
x = None
print(x)  # None

# The None value can be compared with variables.
y = 3
print(y is None)   # False

# The None value cannot be used
# as an argument of arithmetic operators:
# print(None + 7)  # TypeError: unsupported operand ...
```

It is true that the **None** value can not be used as an argument of arithmetic operators. But the **None** value can be assigned and compared to variables. And that can absolutely happen outside of a function.

Question 3:
What is the expected output of the following code?

```
print(1 / 1)
```

A. 1.0
B. 1
C. This can not be predicted.
D. This can not be evaluated.

Answer: A.

Explanation

Topic: `division operator`

```
print(1 / 1)  # 1.0
```

All you have to know here is that the **division operator** always returns a float.Even if it is operating with two integers.

Question 4:
You know that a function named func() resides in a module named mod
The module has been imported using the following line:

```
import mod
```

How can you invoke the function?

A. mod-> func ( )
B. func ( )
C. mod.func ( )
D. mod : : func ( )

Answer: C.

Explanation

Topic: `import`

```
# First execute the following to create the needed file:
text = '''
def func():
    print("I am a function, goo goo g'joob!")
'''
with open('mod.py', 'w') as f:
    f.write(text)

import mod
mod.func()  # I am a function, goo goo g'joob!
```

If you import the whole module you have to add the entity by dot notation.

Question 5:
The += operator, when applied to strings, performs:

A. Multiplication
B. Concatenation
C. Subtraction
D. None of the above.

Answer: B.

Explanation

Topic: `add and assign operator`

```
text = 'Hello'
text += ' '
text += 'World'
print(text)                        # Hello World
print('Hello' + ' ' + 'World')  # Hello World
```

The **add and assign operator** when applied to strings, performs a string concatenation. Just like the **addition operator**

Question 6:
What is the expected output of the following code?

```
data = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16]
]
for i in range(0, 4):
    print(data[i].pop(), end=' ')
```

A. 1 5 9 13
B. 13 14 15 16
C. 1 2 3 4
D. 4 8 12 16

Answer: D.

Explanation

Topics: `for` `range()` `pop()` `print() with end parameter`

```python
data = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16]
]
for i in range(0, 4):
    print(data[i].pop(), end=' ')  # 4 8 12 16

print()
print(list(range(0, 4)))  # [0, 1, 2, 3]

print([1, 2, 3, 4].pop())      # 4
print([5, 6, 7, 8].pop())      # 8
print([9, 10, 11, 12].pop())   # 12
print([13, 14, 15, 16].pop())  # 16
```

data is a two dimensional list.
In every iteration of the **for** loop, **pop()** removes the last element of the corresponding inner list. That last element gets printed. The default value of the **print()** functions end parameter is the line feed. That gets overwritten here by a single space character.

Question 7:
What is the expected output of the following code?

```python
data = {'1': '0', '0': '1'}

for d in data.vals():
    print(d, end=' ')
```

A. 0 0
B. 1 0
C. 0 1
D. The code is erroneous.

Answer: D.

Explanation

Topics: `dictionary` `values()` `for`

```
data = {'1': '0', '0': '1'}

# for d in data.values():
for d in data.vals():  # AttributeError: ...
    print(d, end=' ')    # 0 1
```

This question is a little tricky. Everything seems to be fine, BUT the right name of the dictionary method is **values().**

Question 8:
If you want to import pi from math, which line will you use?

A. from pi import maths
B. import pi from maths
C. from maths import pi
D. None of the above.

Answer: C.

Explanation

Topics: `import` `math` `pi`

```
from math import pi
print(pi)  # 3.141592653589793
```

The order is from big to small. First the module than the variable: From the **math** module import the variable **pi**.

Question 9:
What is the expected output of the following code?

```
try:
    print('try')
except:
    print('except')
finally:
    print('finally')
```

A.	The code is erroneous.
B.	1 | finally 2 | try
C.	1 | finally 2 | except
D.	1 | try 2 | finally
E.	1 | except 2 | finally

Answer: C.

Explanation

Topic: `try` `except` `finally`

```
try:
    print('try')      # try
except:
    print('except')
finally:
    print('finally')  # finally
```

The snippet will try to execute **print('try')** and succeed. The finally block always gets execute.

Question 10:

Which of the following sentences correctly describes the output of the below Python code?

```
data = [4, 2, 3, 2, 1]
res = data[0]

for d in data:
    if d < res:
        res = d

print(res)
```

A. res is the sum of all the numbers in the list.
B. res is the smallest number in the list.
C. res is the largest number in the list.
D. t'es is the average of all the numbers in the list.
E. None of the above.

Answer: B.

Explanation

Topics: `for` `if` `list`

```
data = [4, 2, 3, 2, 1]
res = data[0]
for d in data:
    if d < res:
        print('d in if:', d)  # 2 -> 1
        res = d
print(res)  # 1
```

Classic way to find the smallest number. Take the first element as a possible result. Compare the next number with the possible result. If the next number is smaller, it becomes the new possible result and so forth. In the end the result is the smallest number.

Question 11:

What is the expected output of the following code?

```
print(chr(ord('z') - 2))
```

A. a
B. x
C. z
D. y

Answer: B.

Explanation

Topics: `chr()` `ord()`

```
print(chr(ord('z') - 2))  # x
print(ord('z'))           # 122
print(chr(120))           # x
```

ord() returns an integer representing the Unicode character.
chr() turns that integer back to the Unicode character.
You don't need to remember the number of every character, but like in the alphabet **x** is two before **z**

Question 12:
What is the expected output of the following code?

```
with open('data.txt', 'w') as f:
    f.write("I'm gonna make him an offer he can't refuse.")

with open('data.txt', 'r') as f:
    data = f.readlines()
    for line in data:
        words = line.split()
        print(words)
```

A. I ‘m gonna make him an offer he can’t refuse.
B. [“I’m” , ‘gonna’ , ‘make’ , ‘him’ , ‘an’ , ‘offer’ , ‘he’ , “can;t” , ‘refuse’ .]
C. I
D. The code is erroneous.

Answer: B.

Explanation

Topics: `open()` `write()` `with` `readlines()` `for` `split()`

```
with open('data.txt', 'w') as f:
    f.write("I'm gonna make him an offer he can't refuse.")

with open('data.txt', 'r') as f:
    data = f.readlines()
    print(data)  # ["I'm gonna make him an offer he can't refuse."]
    for line in data:
        words = line.split()
        print(words)
```

The method **readline()** always returns a list. In this case there is only one line in the file and **readline()** returns a list with one element. Therefore the **for** loop only has one iteration. The one element is a string. The **split()** method with no argument passed will split the string at its whitespaces and return a list of the parts. At the end this list gets printed. Actually the **for** loop is superfluous. The following would have sufficed:
print(data[0].split())

Question 13:
Consider the following code.

```
x = 1
x = x == x
```

The value eventually assigned to x is equal to:

A. 0
B. False
C. 1
D. True

Answer: D.

Explanation

Topic: `equal to operator`

```
x = 1
x = x == x
print(x)  # True
x = 1
x = x == x
x = (1 == 1)
x = True
```

x has the value **1**
x gets compared with the **equal to operator** with itself.
It is true, that **x** is equal to **x**
True gets stored in the same variable, in **x**

Question 14:
Consider the following code.

```
from x.y import z
```

The code causes the import of ...

A. entity z from module y from package x
B. entity z from module from package y
C. entity x from module y from package z
D. entity y from module x from package z

Answer: A.

Explanation

Topics: `from` `import`

```python
# First execute the following to create the needed folder and file:
import os
if not os.path.isdir('my_package'):
    os.mkdir('my_package')
text = '''
def my_function():
    print("I am my_function, goo goo g'joob!")
'''
with open('my_package/my_module.py', 'w') as f:
    f.write(text)

from my_package.my_module import my_function
my_function()  # I am my_function, goo goo g'joob!
```

The order is from big to small: **from** package.module **import** entity

Question 15:
What is the expected output of the following code?

```python
data = 'abbabadaadbbaccabc'
print(data.count('ab', 1))
```

A. 2
B. 3
C. 4
D. 5

Answer: A.

Explanation

Topics: `string` `count()`

```python
data = 'abbabadaadbbaccabc'
print(data.count('ab', 1))  # 2
print(data.count('ab', 0))  # 3
```

Remember to bring your best reading glasses to the exam.
The second parameter will determine where the **count()** method will start its counting.
The first parameter (in this case **ab**) is what **count()** will look for.

count() will start at index **1** and therefore it will not find the ab right at the beginning of the string. That leaves two more **ab** to find.

Question 16:
Which of the following statements is false?

A. The result of the / operator is always an integer value.
B. The right argument of the % operator can not be zero.
C. Multiplication precedes addition.
D. The ** operator has right-to-left associativity.

Answer: A.

Explanation

Topic: division operator | integer | float

```
# "The result of the / operator is always an integer value."
# is false:
print(9 / 3)  # 3.0
# The result of the division operator is always a float value!

# "Multiplication precedes addition." is true:
print(3 + 4 * 5)    # 23
print(3 + (4 * 5))  # 23
print(3 + 20)       # 23
print(23)           # 23

# "The ** operator has right-to-left associativity." is true:
print(2 ** 3 ** 2)    # 512
print(2 ** (3 ** 2))  # 512
print(2 ** 9)         # 512
print(512)            # 512

# "The right argument of the % operator cannot be zero."
# is true:
# print(7 % 0)  # ZeroDivisionError: ...
```

As written above, the result of the **division operator** is always a **float** value. Even if it operates with two **integer** values.

Question 17:
What is the expected output of the following code?

```
a = [1, 2, 3, 4, 5]
print(a[3:0:-1])
```

A. [4, 3, 2]
B. [4, 3]
C. The code is erroneous.
D. [4, 3, 2, 1]

Answer: A.

Explanation

Topic: `list slicing`

```
a = [1, 2, 3, 4, 5]
print(a[3:0:-1])  # [4, 3, 2]
```

List slicing: **[start(inclusive):end(exclusive):step]**
The step is negative here. That makes the slice be build from right to left (start interchanges with end). The start is the index **3** (inclusive) and the end is index 0 (exclusive).
Therefore we will get index **3** index **2** and index **1** which are the numbers **4, 3, 2**

Question 18:
What is the expected output of the following code?

```python
people = {}

def add_person(index):
    if index in people:
        people[index] += 1
    else:
        people[index] = 1

add_person('Peter')
add_person('Paul')
add_person('peter')

print(len(people))
```

A. 3
B. The code is erroneous.
C. 2
D. 1

Answer: A.

Explanation

Topics: `def` `shadowing` `dictionary`

```python
people = {}

def add_person(index):
    if index in people:
        people[index] += 1
    else:
        people[index] = 1

add_person('Peter')
add_person('Paul')
add_person('peter')

print(len(people))  # 3
print(people)       # {'Peter': 1, 'Paul': 1, 'peter': 1}
```

Peter is a different index than **peter** because **P** is a different character than **p** and therefore the dictionary will have three entries.

Question 19:
You develop a Python application for your company. A list named employees contains 200 employee names, the last five being company management. You need to slice the list to display all employees excluding management. Which code segments can you use? Choose two.

A. employees [: -5]
B. employees [0: -5]
C. employees [1: -5]
D. employees [0: -4]
E. employees [1: -4]

Answer: A, B.

Explanation

Topic: `list slicing`

```python
employees = []

# for i in range(1, 196):
for i in range(1, 6): # Just for convenience
    employees.append('Employee' + str(i))

for i in range(1, 6):
    employees.append('Manager' + str(i))

print(employees)
print(employees[:-5])
print(employees[0:-5])
print(employees[1:-4])
# One manager present and one employee is missing
print(employees[1:-5])  # One employee is missing
print(employees[0:-4])  # One manager present
```

List slicing: **[start(inclusive):end(exclusive)]** .
The default value for the start is **0.** Meaning you can write it or leave it out. The negative index counts from the end. The end is exclusive.
-1 cuts out one element. **-5** cuts out five elements. And that is what you need here for the five managers.

Question 20:
What is the expected output of the following code?

```python
x = 9
y = 12
result = x // 2 * 2 / 2 + y % 2 ** 3
print(result)
```

A. 7.0
B. 8.0
C. 9.0
D. 8

Answer: B.

Explanation

Topics: arithmetic operators | operator precedence

```
x = 9
y = 12
result = x // 2 * 2 / 2 + y % 2 ** 3
print(result)                          # 8.0

print(x // 2 * 2 / 2 + y % 2 ** 3)     # 8.0

print(9 // 2 * 2 / 2 + 12 % 2 ** 3)    # 8.0

print(9 // 2 * 2 / 2 + 12 % (2 ** 3))  # 8.0
print(9 // 2 * 2 / 2 + 12 % 8)         # 8.0

print((9 // 2) * 2 / 2 + 12 % 8)       # 8.0
print(4 * 2 / 2 + 12 % 8)              # 8.0

print((4 * 2) / 2 + 12 % 8)            # 8.0
print(8 / 2 + 12 % 8)                  # 8.0

print((8 / 2) + 12 % 8)                # 8.0
print(4.0 + 12 % 8)                    # 8.0

print(4.0 + (12 % 8))                  # 8.0
print(4.0 + 4)                         # 8.0

print(8.0)                             # 8.0
```

The operators here are from three different groups:
"Exponent" has the highest precedence. Followed by "Multiplication, Division, Floor division, Modulus". "Addition, Subtraction" has the lowest precedence. Therefore the order of operations here is: ****** -> // -> ***** -> / -> **%** -> **+**

Question 21:
What is the expected output of the following code?

```
data = [[0, 1, 2, 3] for i in range(2)]
print(data[2][0])
```

A. 2
B. 1
C. 0
D. The code is erroneous.

Answer: D.

Explanation

Topics: `list comprehension` `list indexing` `IndexError`

```
data = [[0, 1, 2, 3] for i in range(2)]
print(data[2][0])  # IndexError: list index out of range

print(data)     # [[0, 1, 2, 3], [0, 1, 2, 3]]
print(data[0])  # [0, 1, 2, 3]
print(data[1])  # [0, 1, 2, 3]
```

range(2) has two elements: **0** and **1.** Therefore the outer list will have two elements. And **data[2]** does not exist.

Question 22:
What is the expected output of the following code?

```
data = set([1, 2, 2, 3, 3, 3, 4, 4, 4, 4])
print(len(data))
```

A. 0
B. 1
C. 4
D. 2
E. 3
F. 10

Answer: C.

Explanation

Topics: `list` `set()` `len()`

```
data = set([1, 2, 2, 3, 3, 3, 4, 4, 4, 4])
print(len(data))  # 4

print(type(data))  # <class 'set'>
print(data) # {1, 2, 3, 4}

# Also works directly:
print(len({1, 2, 2, 3, 3, 3, 4, 4, 4, 4}))  # 4
```

A set can not have duplicates.

Question 23:
Consider the following code.

```
import random
data = [10, 20, 30]
random.shuffle(data)
print(data)
```

Which of the following statements best describes the behaviour of the random.shuffle() method?

A. It shuffles the elements for the number of times equal to the size of the list.
B. It will not modify the list. This function is just a placeholder and yet to be implemented.
C. It shuffles the elements of the list in-place.
D. It returns a list where the elements 10, 20, and 30,would be at random positions.

Answer: C.

Explanation

Topics: `random.shuffle()` `list`

```python
import random
data = [10, 20, 30]
random.shuffle(data)
print(data)  # e.g. [20, 10, 30]

print(random.shuffle(data))  # None
```

The original list gets shuffled in-place. The return value is **None**

Question 24:
A code point is:

A. A point used to write a code.
B. Acode containing a point.
C. A number which makes up a character.
D. None of the above.

Answer: C.

Explanation

Topic: `code points`

```python
print(ord('A')) # 65
```

For example in ASCII the number **65** makes up the character **A**
https://en.wikipedia.org/wiki/Code_point

Question 25:
The following statement ...

```python
assert x == 0
```

A. will stop the program if x is not equal to 0
B. has no effect.
C. will stop the program if x is equal to 0
D. is erroneous.

Answer: A.

Explanation

Topic: `assert` `AssertionError`

```
x = 0
assert x == 0
print('Hello')  # Hello

x = 7
assert x == 0   # AssertionError
print('Hello')
```

If the assertion (here **x == 0)** is **True** the program will continue. Otherwise an **AssertionError** occurs.

Question 26:
What is the expected output of the following code?

```
print(1 // 2 * 3)
```

A. 4.5
B. 0
C. 0.0
D. 0.16666666666666666

Answer: B.

Explanation

Topics: `arithmetic operators` `operator precedence`

```
print(1 // 2 * 3)    # 0

print((1 // 2) * 3)  # 0
print(0 * 3)         # 0
print(0)             # 0
```

There are only operators from the group "Multiplication, Division, Floor division, Modulus". The group has a left-to-right associativity, meaning the left one gets evaluated first. Therefore the order of operations here is: // -> *

Question 27:

The part of your code where the handling of an exception takes place should be placed inside:

A. the try: branch
B. the except: branch
C. the exception: branch
D. None of the above.

Answer: B.

Explanation

Topic: `except`

```
try:
    # number = input('Please enter a number\n')
    number = 'one'
    zahl = int(number)
    print('Perfekt!')
except:
    print('Something went wrong.')
```

It is about exceptions, but the branch is called **except:**

Question 28:
Consider the following code.

```
for n in range(1, 6, 1):
    print(??? * 5)
```

What would you insert instead ??? so that the program prints the following pattern to the monitor?

```
11111
22222
33333
44444
55555
```

A. n
B. str(n)

C. -1
D. 1
E. 2

Answer: B.

Explanation
Topics:

`for`	`range()`	`str()`	`multiply operator string concatenation`

```python
for n in range(1, 6, 1):
    print(str(n) * 5)
"""
11111
22222
33333
44444
55555
"""

print('----------')

for n in range(1, 6, 1):
    print(n * 5)
"""
5
10
15
20
25
"""

print('----------')
```

```
24
25  for n in range(1, 6, 1):
26      print(-1 * 5)
27  """
28  -5
29  -5
30  -5
31  -5
32  -5
33  """
34
35  print('----------')
36
37  for n in range(1, 6, 1):
38      print(1 * 5)
39  """
40  5
41  5
42  5
43  5
44  5
45  """
46
47  print('----------')
```

```

for n in range(1, 6, 1):
    print(2 * 5)
"""
10
10
10
10
10
"""
```

range(1, 6, 1) delivers the right numbers 1, 2, 3, 4, 5. (It would also work without step **1**, because that's the default value.) You need the **str()** function here to get more numbers. Otherwise a calculation takes place and you end up with one number per row. By string concatenation you get the right result:

```
'1' * 5 -> '11111'
'2' * 5 -> '22222'
'3' * 5 -> '33333'
'4' * 5 -> '44444'
'5' * 5 -> '55555'
```

Question 29:
How many stars will the following code print to the monitor?

```
x = 1
while x < 10:
    print('*')
    x = x << 1
```

A. one
B. eight
C. four
D. two

Answer: C.

Explanation

Topics: `while` `bit operator`

```
x = 1
while x < 10:
    print('*')
    x = x << 1
    print('x:', x)            # 2 -> 4 -> 8 -> 16
    print('bin(x):', bin(x))  # 0b10->0b100->0b1000->0b10000

"""
1 -> 00001 << 1 -> 00010 (2)
2 -> 00010 << 1 -> 00100 (4)
4 -> 00100 << 1 -> 01000 (8)
8 -> 01000 << 1 -> 10000 (16)
"""
```

Left shift by one, a classic way to double values. Every value goes to the bit on its left and thereby doubles in value. And **1, 2, 4, 8** are all smaller than **10** but not the **16** and therefore four stars will be printed.

Question 30:
UTF-8 is ...

A. an encoding form of the Unicode Standard.
B. the 9th version of the UTF Standard.
C. a Python version name.
D. a synonym for “byte’

Answer: A.

Explanation

Topic: `UTF-8`

UTF-8 is one of three encoding forms of the Unicode Standard. The others are **UTF-16** and **UTF-32**. UTF-8 is the most popular one, because it is the most flexible. **UTF-8** requires 8, 16, 24 or 32 bits (one to four bytes) to encode a Unicode character, **UTF-16** requires either 16 or 32 bits to encode a character, and **UTF-32** always requires 32 bits to encode a character.

Question 31:
Which of the following function headers is correct?

A. def func (a=1, b=1, c=2):
B. def func (a =1, b):
C. def func (a=1, b=1, c=2, d):
D. def func (a=1,b, c=2):

Answer: A.

Explanation

Topics: `def` `default arguments`

```
def func(a=1, b=1, c=2): pass

# def func(a=1, b): pass            # SyntaxError: ...
# def func(a=1, b, c=2): pass       # SyntaxError: ...
# def func(a=1, b=1, c=2, d): pass  # SyntaxError: ...

# This would also work:
def func(a, b=1, c=2): pass

"""
def func(a=1, b): pass
func(7)
The 7 would go into a and there is nothing left for b.
"""
```

The default argument(s) have to be at the end.

Question 32:
What is the expected output of the following code?

```
colors = ['red\n', 'yellow\n', 'blue\n']
file = open('wellert.txt', 'w+')
file.writelines(colors)
file.close()
file.seek(0)
for line in file:
    print(line)
```

A.	redyellowblue
B.	The code is erroneous.
C.	[‘red\n’ , ‘yellow\n’ , ‘blue\n’]
D.	1 | red 2 | 3 | yellow 4 | 5 | blue

Answer: B.

Explanation

Topics: `open()` `writelines()` `seek()` `close()` `for`

```
colors = ['red\n', 'yellow\n', 'blue\n']
file = open('wellert.txt', 'w+')
file.writelines(colors)
file.close()
file.seek(0)  # ValueError: I/O operation on closed file.
for line in file:
    print(line)
file.close()
```

The file gets closed too early. All operations on the file need to be finished before you can close it.

Question 33:
What is the expected output of the following code?

```python
import os

os.mkdir('thumbnails')
os.chdir('thumbnails')

sizes = ['small', 'medium', 'large']

for size in sizes:
    os.mkdir(size)

print(os.listdir())
```

A. [‘large’ , ‘medium’ , ‘small’]
B. [ ]
C. [‘.’ , ‘. . ‘ ‘large’ , ‘medium’ , ‘small’]
D. [‘.’ , ‘large’ , ‘medium’ , ‘small’]

Answer: A.

Explanation

Topics: `import` `os` `mkdir()` `chdir()` `list` `for` `listdir()`

```python
import os

# To be able to run the file more often
# you have to remove the 'thumbnails' directory:
import shutil
shutil.rmtree('thumbnails')

os.mkdir('thumbnails')
os.chdir('thumbnails')

sizes = ['small', 'medium', 'large']

for size in sizes:
    os.mkdir(size)

print(os.listdir())  # ['large', 'medium', 'small']
```

This is pretty straight forward. You make the directory **thumbnails** you change into it, you make the three directories **small medium large** and then you list them. But you need the knowledge, that **listdir()** never includes the special entries **.** and **..**
https://www.tutorialspoint.com/python/os_listdir.htm

Question 34:
Which of the following items are present in the function header?

A. function name and parameter list
B. return value
C. function name
D. parameter list

Answer: A.

Explanation

Topics: `def` `parameter` `function header`

```python
def func(para1, para2):  # This line is the function header
    para1 + para2
```

In the function header is the keyword **def**

the function name and the parameter list.

Question 35:
What is the expected output of the following code?

```
try:
    raise Exception(1, 2, 3)
except Exception as e:
    print(len(e.args))
```

A. 1
B. The code is erroneous.
C. 2
D. 3

Answer: D.

Explanation

Topics: `try` `except` `args` `len()` `Exception`

```
try:
    raise Exception(1, 2, 3)
except Exception as e:
    print(len(e.args))   # 3
    print(type(e.args))  # <class 'tuple'>
    print(e.args)        # (1, 2, 3)

try:
    x = 1 / 0
except Exception as e:
    print(len(e.args))  # 1
    print(e.args)       # ('division by zero',)
```

If you raise an **Exception** you can pass a list of arguments. They will be assigned as a tuple to the **args** attribute of the **Exception** object.

Question 36:
What is the expected output of the following code?

```
x = """
"""
print(len(x))
```

A. 2
B. 1
C. 0
D. The code is erroneous.

Answer: B.

Explanation

Topics: `multiline string` `len()`

```
x = """
"""
print(len(x))  # 1

# ord() returns an integer representing the Unicode character.
print(ord(x[0]))  # 10 (LF: line feed, new line)

# Same result with single quotes:
y = '''
'''
print(len(y))  # 1

# Every line feed is a character:
z = """

"""
print(len(z))  # 2
```

In a multiline string the line feed gets saved like any other character.

Question 37:
What is the expected output of the following code?

```python
data = [1, 2, 3, 4, 5, 6]

for i in range(1, 6):
    data[i - 1] = data[i]

for i in range(0, 6):
    print(data[i], end=' ')
```

A. 1 2 3 4 5 6
B. 2 3 4 5 6 6
C. 1 1 2 3 4 5
D. 2 3 4 5 6 1

Answer: B.

Explanation

Topics: `for` `range()` `print() with end parameter` `list indexing`

```python
data = [1, 2, 3, 4, 5, 6]

for i in range(1, 6):  # 1 -> 2 -> 3 -> 4 -> 5
    data[i - 1] = data[i]

print(data)  # [2, 3, 4, 5, 6, 6]

# This is just for output:
for i in range(0, 6):
    print(data[i], end=' ')  # 2 3 4 5 6 6
```

In the first **for** loop, the five last values of **data** get written to the first five indexes of **data**
The value of the last index stays the same. **range(1, 6)** will produce the numbers **1, 2, 3, 4, 5**
And because the first of these number is **1** we need the **data[i -1]** to start at index **0**.
The old value at index **1** is **2** .That gets written at index **0**
And so forth ...

Question 38:
What is the expected output of the following code if the user enters 11 and 4 ?

```
x = int(input())
y = int(input())
x = x % y
x = x % y
y = y % x
print(y)
```

A. 2
B. 3
C. 4
D. 1

Answer: D.

Explanation

Topics: `input()` `modulus operator`

```
# x = int(input())  # Input: 11
# y = int(input())  # Input: 4
x, y = 11, 4    # Just for convenience
x = x % y
print(11 % 4)  # 3
x = x % y
print(3 % 4)   # 3
y = y % x
print(4 % 3)   # 1
print(y)       # 1
```

input() returns strings, but the **int()** function casts them to integers. Then you have to concentrate with all the modulus operators.

Question 39:
What is the expected result of the following code?

```
b = bytearray(3)
print(b)
```

A. 3
B. bytearray (b ‘3’)
C. bytearray (0, 0, 0)
D. bytearray (b’ \x00\x00\x00’)

Answer: D.

Explanation

Topic: `bytearray()`

```
b = bytearray(3)
print(b)  # bytearray(b'\x00\x00\x00')

# The bytearray works better with sequences of integers:
print(bytearray([3]))           # bytearray(b'\x03')
print(bytearray([65, 66, 67]))  # bytearray(b'ABC')
```

Read more about it here:
https://www.programiz.com/python-programming/methods/built-in/bytearray

Question 40:
What is the expected output of the following code?

```
x = '\\\\'
print(len(x))
```

A. 1
B. The code is erroneous.
C. 2
D. 4

Answer: C.

Explanation

Topics: `escaping` `len()`

```
# print(len('\'))    # SyntaxError: ...
print(len('\\'))     # 1
# print(len('\\\'))  # SyntaxError: ...
print(len('\\\\'))   # 2
```

The backslash is the character to escape another character. If you write '\' the backslash escapes the ending single quote to a normal character. It takes its syntactical meaning and the single quote becomes a normal character and it loses its ability to end the string. And therefore we get the syntax error. If you write '\\' the one backslash escapes the other and you end up with a string with one normal backslash. If you write '\\\\' it is kind of the same and you end up with a string with two normal backslashes.

Question 41:
What is the expected behaviour of the following snippet?

```
1  x = 1
2
3
4  def a(x):
5      return 2 * x
6
7
8  x = 2 + a(x)      # Line 8
9  print(a(x))       # Line 9
```

It will:

A. print 4
B. print 6
C. cause a runtime exception on Line 8
D. print 8
E. cause a runtime exception on Line 9

Answer: D.

Explanation

Topics: `def` `return` `multiplication operator` `addition operator`

```
1   x = 1
2
3
4   def a(x):
5       return 2 * x
6
7
8   x = 2 + a(x)      # Line 8
9   print(a(x))       # Line 9
10  # 8
```

The first call of the function **a()** will pass **1** to the function. **2 * 1 -> 2**
The function will return 2
2 + 2 -> 4
The second call of the function a() will pass 4 to the function.
2 * 4 -> 8
8 will be printed.

Question 42:
What is the expected output of the following code?

```
1   data = 'abcdefg'
2
3   def func(text):
4       del text[2]
5       return text
6
7   print(func(data))
```

A. abcef
B. acdef
C. abdef
D. The code is erroneous.

Answer: D

Explanation

Topics: `def` `del` `string (immutable)` `string indexing`

```
data = 'abcdefg'

def func(text):
    # del text[2]  # TypeError: ...
    return text

print(func(data))

# Strings are immutable
# The indexes are readable but not writeable:
s = 'Hello'
print(s[0])   # H
# s[0] = 'h'  # TypeError: ...
```

A string is immutable. You can not change it. You can read something by indexing, BUT you can not write something by indexing.

Question 43:
Which of the following function calls can be used to invoke the below function definition?

```
def test(a, b, c, d):
```

Choose three.

A. test (1, 2, 3, d=4)
B. test (a=1, b=2, c=3, d=4)
C. test (a=1, 2, 3, 4)
D. test (a=1, b=2, c=3, 4)
E. test (1, 2, 3, 4)
F. test (a=1,2, c=3, 4)

Answer: A, B, E.

Explanation

Topics: `def` `positional/keyword arguments`

```python
def test(a, b, c, d):
    print(a, b, c, d)

test(1, 2, 3, 4)          # 1 2 3 4
test(a=1, b=2, c=3, d=4)  # 1 2 3 4
test(1, 2, 3, d=4)        # 1 2 3 4

# test(a=1, 2, 3, 4).     # SyntaxError: ...
# test(a=1, b=2, c=3, 4)  # SyntaxError: ...
# test(a=1, 2, c=3, 4)    # SyntaxError: ...

# print(end='', 'Hello')  # SyntaxError: ...
```

The keyword arguments always have to be at the end.

Question 44:
How many stars will the following code send to the monitor?

```python
x = 0
while x < 6:
    x += 1
    if x % 2 == 0:
        continue
    print('*')
```

A. two
B. three
C. one
D. zero

Answer: B.

Explanation

Topics: `while` `if` `continue`

```
x = 0
while x < 6:
    print('1. x:', x)         # 0 -> 1 -> 2 -> 3 -> 4 -> 5
    x += 1
    print('2. x:', x)         # 1 -> 2 -> 3 -> 4 -> 5 -> 6
    if x % 2 == 0:
        print('x in if:', x)  # 2 -> 4 -> 6
        continue
    print('x behind if:', x)  # 1 -> 3 -> 5
    print('*')
"""
*
*
*
"""
```

When **x** is **2**, **4** and **6** the **if** condition is **True** and the **continue** gets triggered. The iteration ends directly and the **print()** function doesn't get executed. When **x** is **1**, **3** and **5** the **if** condition is **False** the **continue** does not get triggered and those three times the **print()** function gets executed. Therefore there will be three stars printed.

Question 45:
What is the expected output of the following code?

```
def func(message, num=1):
    print(message * num)

func('Hello')
func('Welcome', 3)
```

A.	1 | Hello 2 | Welcome Welcome Welcome
B.	1 | Hello 2 | WelcomeWelcomeWelcome
C.	1 | Hello

	2 | Viewers
D.	1 | Hello 2 | Welcome, Welcome, Welcome

Answer: B.

Explanation
Topics:

`def` `default parameter` `multiply operator string concatenation`

```
def func(message, num=1):
    print(message * num)

func('Hello')       # Hello
func('Welcome', 3)  # WelcomeWelcomeWelcome
```

In the function a string concatenation by multiplication takes place. Once with the default value of **num** (**1**) and once with the one passed by argument **(3)**

Question 46:
What is the expected output of the following code?

```
x = True
y = False
x = x or y
y = x and y
x = x or y
print(x, y)
```

A. False False
B. False True
C. True True
D. True False

Answer: D.

Explanation
Topics: `logical operators` `boolean`

```
x = True
y = False
x = x or y   # True or False -> True
y = x and y  # True and False -> False
x = x or y   # True or False -> True
print(x, y)  # True False
```

Nothing changes here. The variables always get assigned the value they had before.

Question 47:
What is the expected output of the following code?

```
print('Peter' 'Wellert')
```

A. The code is erroneous.
B. Wellert
C. PeterWellert
D. Peter

Answer: C.

Explanation

Topic: `string concatenation`

```
print('Peter' 'Wellert')  # PeterWellert

x = 'Hello' 'world'
print(x)  # Helloworld
```

String literals that are delimited by whitespace are automatically concatenated.

Question 48:
Which of the following variable names is illegal?

A. tRUE
B. TRUE
C. True
D. true

Answer: C.

Explanation

Topic: `naming variables`

```
# True = 3  # SyntaxError: cannot assign to True
# Those three work, because Python is case sensitive:
true = 7
tRUE = 23
TRUE = 42
print(true, tRUE, TRUE)  # 7 23 42
```

You can not name a variable like a Python keyword.

Question 49:
What is the expected output of the following code?

```
print(float('1.3'))
```

A. 13
B. 1.3
C. The code is erroneous.
D. 1, 3

Answer: B.

Explanation

Topic: `float()`

```
print(float('1.3'))  # 1.3
```

The function **float()** does its normal job here and converts the string to a float.

Question 50:
How many stars will the following code print to the monitor?

```
i = 0
while i < i + 2:
    i += 1
    print('*')
else:
    print('*')
```

A. The snippet will enter an infinite loop.
B. two
C. zero
D. one

Answer: A.

Explanation

Topics: `while` `else (nobreak)`

```
i = 0
while i < i + 2:
    i += 1
    print('*')
    if i == 1000: break  # Safeguard
else:
    print('*')
```

i gets incremented inside the **while** loop,
BUT **i** will always be smaller than **i + 2**
Therefore the **while** condition will always be **True** and we build ourselves a nice infinite loop.

Question 51:
What is the expected output of the following code?

```
x = 1 + 1 // 2 + 1 / 2 + 2
```

```
print(x)
```

A. 4.0
B. 4
C. 3
D. 3.5

Answer: D.

Explanation

Topics: arithmetic operators | operator precedence

```
x = 1 + 1 // 2 + 1 / 2 + 2
print(x)                        # 3.5

print(1 + 1 // 2 + 1 / 2 + 2)    # 3.5

print(1 + (1 // 2) + 1 / 2 + 2)  # 3.5
print(1 + 0 + 1 / 2 + 2)         # 3.5

print(1 + 0 + (1 / 2) + 2)       # 3.5
print(1 + 0 + 0.5 + 2)           # 3.5

print((1 + 0) + 0.5 + 2)         # 3.5
print(1 + 0.5 + 2)               # 3.5

print((1 + 0.5) + 2)             # 3.5
print(1.5 + 2)                   # 3.5

print(3.5)                       # 3.5
```

The operators here are from two different groups:
The group "Multiplication, Division, Floor division, Modulus" has a higher precedence than the group "Addition, Subtraction".
Therefore the order of operations here is: // -> / -> + -> + -> +

Question 52:
What is the expected output of the following code?

```python
data = {}

def func(d, key, value):
    d[key] = value

print(func(data, '1', 'Peter'))
```

A. None
B. value
C. Peter
D. 1
E. The code is erroneous.

Answer: A.

Explanation

Topics: `def` `return`

```python
data = {}

def func(d, key, value):
    d[key] = value

print(func(data, '1', 'Peter'))  # None
```

func() has no return statement. Therefore **None** gets returned.

Question 53:
The ABC Video company needs a way to determine the cost that a customer will pay for renting a DVD. The cost is dependent on the time of day the DVD is returned. However, there are also special rates on Thursdays and Sundays. The fee structure is shown in the following list:
The cost is $1.59 per night. If the DVD is returned after 8 PM, the customer will be charged an extra day. If the video is rented on a Sunday, the customer gets 30% off for as long as they keep the video. If the video is rented on a Thursday, the customer gets 50% off for as long as they keep the video.
You need to write code to meet the requirements.

```python
# ABC Video, DVD Rental Calculator

ontime = input('Was the video returned before 8 pm? y or n').lower()
days_rented = int(input('How many days was the video rented?'))
day_rented = input('What day was the video rented?').capitalize()

cost_per_day = 1.59

if ontime XXX
    days_rented += 1
if day_rented YYY
    total = (days_rented * cost_per_day) * .7
elif day_rented ZZZ
    total = (days_rented * cost_per_day) * .5
else:
    total = (days_rented * cost_per_day)

print('Cost of the DVD rental is: $', total)
```

What should you insert instead of XXX, YYY and ZZZ?

A.	1 | XXX -> == ‘y’ : 2 | YYY -> == ‘Sunday’ : 3 | ZZZ -> == ‘Thursday’ :
B.	1 | XXX -> != ‘n’ : 2 | YYY -> is ‘Sunday’ : 3 | ZZZ -> is ‘Thursday’ :
C.	1 | XXX -> == ‘n’: 2 | YYY -> is ‘Sunday’ : 3 | ZZZ -> is ‘Thursday’ :
D.	1 | XXX -> == ‘n’ : 2 | YYY -> ‘Sunday’ : 3 | ZZZ -> ‘Thursday’ :
E.	1 | XXX -> ‘y’ : 2 | YYY -> ‘Sunday’: 3 | ZZZ -> ‘Thursday’ :

Answer: D.

Explanation

Topics: `input()` `int()` `if` `elif` `else`

```python
# ABC Video, DVD Rental Calculator

# ontime = input('Was the video returned before 8pm? y or n').lower()
ontime = 'y'
# ontime = 'n'

# days_rented = int(input('How many days was the video rented?'))
days_rented = int('4')

# day_rented = input('What day was the video rented?').capitalize()
day_rented = 'sunday'.capitalize()
# day_rented = 'thursday'.capitalize()
# day_rented = 'monday'.capitalize()

cost_per_day = 1.59

if ontime == 'n':
    days_rented += 1
if day_rented == 'Sunday':
    total = (days_rented * cost_per_day) * .7
elif day_rented == 'Thursday':
    total = (days_rented * cost_per_day) * .5
else:
    total = (days_rented * cost_per_day)

print('Cost of the DVD rental is: $', total)
# n, 4, monday   -> 7.95
# y, 4, monday   -> 6.36
# y, 4, thursday -> 3.18
# y, 4, sunday   -> 4.452
```

At this company to return a video on time means to return it before 8 PM.
Otherwise one day is added to the amount of rented days. If you return your video on a Sunday you get 30% off. Meaning you only pay 70%.
And if you return your video on a Thursday, you pay only half (50%).

Question 54:
Which of the following functions immediately terminates a program?

A. None
B. sys.halt ( )
C. sys.exit ( )
D. sys.stop ( )
E. sys.terminate ( )

Answer: C.

Explanation

Topic: `sys.exit()`

```
import sys
print('Before the exit!')  # Before the exit!
sys.exit()
# sys.halt()      # AttributeError: ...
# sys.stop()      # AttributeError: ...
# sys.terminat()  # AttributeError: ...
print('After the exit!')
```

The method in Python to immediately terminate a program is **sys.exit()**
The others methods do not exist.

Question 55:
Which of the following is false?

A. A try statement can have one or more final clauses.
B. A try statement can have a finally clause and an except clause.
C. A try statement can have one or more except clauses.
D. A try statement can have a final clause without an except clause.

Answer: A.

Explanation

Topics: `try` `except` `finally`

```python
# A try statement can have a finally clause and an except clause.
try:
    print('try')      # try
except:
    print('except')
finally:
    print('finally')  # finally

# A try statement can have a finally clause without an except clause.
try:
    print('try')      # try
finally:
    print('finally')  # finally

# A try statement can have one or more except clauses.
try:
    # value = input('Please enter a number!')
    value = 'one'
    # value = '0'
    number = 100 / int(value)
except ValueError:
    print('Please enter an integer!')
except ZeroDivisionError:
    print('Please do not enter zero!')
else:
    print('Result:', number)

"""
# A try statement can have one or more finally clauses.
try:
    print('try')
finally:
    print('finally 1')
finally:                # SyntaxError: invalid syntax
    print('finally 2')
"""
```

Yes, a **try** statement can have one or more **except** clauses.
Yes, a **try** statement can have a **finally** clause and an except clause.
Yes, a **try** statement can have a **finally** clause without an **except** clause.
But NO, a **try** statement can NOT have more **finally** clauses.

Question 56:
What value will be assigned to the x variable?

```
z = 3
y = 7
x = y == z and y > z or z > y and z != y
```

A. True
B. 0
C. 1
D. False

Answer: D.

Explanation
Topics:

logical operators | relational operators | operator precedence

```
z = 3
y = 7
x = y == z and y > z or z > y and z != y     # False
print(x)                                     # False
print(7 == 3 and 7 > 3 or 3 > 7 and 3 != 7)  # False
print(False and True or False and True)      # False
print((False and True) or (False and True))  # False
print(False or False)                        # False
print(False)                                 # False
```

The operators here are from three different groups. "Comparisons, Identity, Membership operators", "Logical AND", "Logical OR". The three different comparison operators **greater than operator equal to operator not equal to operator** have the highest precedence. Then the **logical and operator** has a higher precedence than the **logical or operator**

Question 57:
What is the expected output of the following code?

```python
num = 1

def func():
    num = 3
    print(num, end=' ')

func()

print(num)
```

A. 1 1
B. 1 3
C. 3 3
D. The code is erroneous.
E. 3 1

Answer: E.

Explanation

Topics: `def` `print() with end parameter`

```python
num = 1

def func():
    num = 3              # Shadows name 'num' from outer scope
    print(num, end=' ')  # 3

func()

print(num)  # 1
```

The **num** variable inside the function scope will be a different variable. It will shadow the name of the **num** variable from the outer scope, but it will be a different entity.